SMBFlow: A Deterministically Governed Multi-Agent Architecture for Context-Aware Workflow Automation in Enterprise Environments

Hemantkumar S. Lakhane*, Tanmay Kumbhare*, Varsha Dange*, Dhanashri Khode*, and Khushi Ghumre*
*Department of Artificial Intelligence and Data Science / Machine Learning
Vishwakarma Institute of Technology, Pune, 411037, Maharashtra, India
Email: {hemantkumar.lakhane24, tanmay.kumbhare24, varsha.dange1, dhanashri.khode24, khushi.ghumre24}@vit.edu

Abstract—Enterprises increasingly deploy Large Language Model (LLM) multi-agent systems to automate mission-critical workflows across fragmented software ecosystems. However, unconstrained stochastic agents introduce catastrophic failure modes: hallucinated state transitions, unauthorized side-effects, prompt injection vulnerabilities, and absent cryptographic auditability. In this paper, we present SMBFlow, an enterprise-grade multi-agent workflow architecture grounded in a strict principle: probabilistic agents propose, deterministic gates decide. SMBFlow enforces capability-based execution isolation where agents hold zero tool credentials, requiring cryptographically signed, single-use HMAC execution tokens issued only after typed policy predicates and calibrated risk scoring are satisfied. We implement conformal risk calibration bounding false executions, adaptive human-in-the-loop (HITL) thresholding, Saga-style backward compensation, hybrid GraphRAG context retrieval combining dense embeddings with Personalized PageRank, and a quality-constrained cost optimizer. Furthermore, SMBFlow hardens execution provenance through a Merkle-tree Evidence and Provenance Infrastructure (EPI) with periodic signed external anchoring, providing formal computational resistance against retroactive tampering and truncation. We conduct an extensive empirical evaluation across three multi-tenant enterprise workflows (900 full runs, 240 adversarial attack vectors, and 1,500 calibration trials). SMBFlow achieves a 0.0% unsafe execution rate (compared to 14.7%–20.3% for ungated ReAct and 8.0%–8.7% for LangGraph guardrails), reduces token expenditure by 51.8% relative to frontier models ($p < 10^{-50}$, Cohen's $d = 10.06$), resists 100% of adversarial attacks, and verifies 50,000 execution steps in 704.8 ms while adding merely 2.21 ms of non-LLM governance overhead.

Index Terms—Multi-Agent Systems, Workflow Orchestration, Deterministic Governance, Capability Security, Conformal Risk Calibration, GraphRAG, Cryptographic Provenance.

I. Introduction

Small and medium-sized enterprises (SMEs) face acute operational bottlenecks due to fragmented Software-as-a-Service (SaaS) applications, manual cross-platform data synchronization, and lean technical resources [1]. While modern businesses rely on diverse platforms—CRM tools, billing gateways, ERP databases, and messaging channels—these systems lack autonomous interoperability.

Recent advancements in Large Language Model (LLM) agents and multi-agent architectures [2]–[8] offer substantial promise for enterprise workflow automation. By decomposing complex workflows into specialized agent personas, multi-agent frameworks parse unstructured business documents, reason across relational records, and formulate multi-step operational plans.

However, deploying unconstrained stochastic agents in production enterprise environments introduces prohibitive risks: (1) Stochastic State Mutation: Generative LLMs exhibit probabilistic drift and hallucination, generating invalid tool parameters (e.g., erroneous ledger modifications or unauthorized refunds) [9]; (2) Prompt Injection: Indirect prompt injections embedded in external data (e.g., invoices, emails) hijack agent reasoning, leaking tenant data or evading safety prompts [10], [11]; (3) Credential Over-Privilege: Conventional frameworks equip agent personas with ambient API keys, enabling compromised agents to execute external side effects directly [12]; (4) Absence of Cryptographic Provenance: Existing frameworks log state mutations into mutable tables without cryptographic integrity proofs, failing regulatory compliance standards (e.g., EU AI Act [13]) requiring tamper-evident lineage [14], [15]; (5) Prohibitive Operational Expenditure: Universal routing to frontier models incurs unsustainable inference expenses [16], [17].

To resolve these fundamental challenges, we introduce SMBFlow, an enterprise-grade multi-agent workflow architecture grounded in a strict systems invariant: probabilistic LLM agents propose actions, but deterministic policy engines and cryptographic execution gates decide and execute.

Contributions: The core contributions of this work are:

- Capability-Based Execution Isolation: We design an execution plane where agents possess zero tool credentials. External connectors accept invocations if and only if accompanied by a single-use, HMAC-signed execution token issued exclusively by a deterministic governance gate after static policy checking and risk verification. We formally state and prove the execution isolation invariant.

- Calibrated Risk Scoring and Adaptive HITL: We replace heuristic risk weights with Platt-calibrated scoring bounded by conformal false-execute limits ($\leq 5\%$), paired with an online adaptive Human-in-the-Loop (HITL) threshold controller that reduces operator review load while maintaining safety.
- Hardened Merkle Provenance (EPI): We construct an Evidence and Provenance Infrastructure (EPI) combining Merkle-tree batching with signed external checkpoint anchoring. We formally prove computational immutability under collision resistance (Theorem 1) and truncation detection under external anchoring (Theorem 2).
- Hybrid GraphRAG and Constrained Routing: We formulate a multi-hop context retrieval engine fusing dense vector similarity with Personalized PageRank (PPR), alongside a quality-constrained cost optimizer reducing inference expenditure by 51.8% ($p < 10^{-50}$).
- Comprehensive Empirical Validation: We construct a reproducible testbed spanning three multi-tenant enterprise workflows (900 full runs, 240 red-team attack vectors, 1,500 calibration trials). We measure latency, cost, safety, scalability to 100 concurrent tenants, and verification scaling to 50,000 steps.

II. Related Work and Gap Analysis

A. Multi-Agent Frameworks and Tool-Use Safety

Autonomous LLM agents leverage cognitive paradigms such as ReAct [2] and Toolformer [3] to interleave reasoning with tool execution. Multi-agent systems such as AutoGen [4], MetaGPT [6], and Generative Agents [18] coordinate agents via natural language dialogue. LangGraph [5] introduces cyclic state-graph scheduling. However, as demonstrated by ToolEmu [9], conversational multi-agent systems suffer from cascading hallucination and lack deterministic execution containment. Prompt-level guardrails (e.g., Llama Guard [19], NeMo Guardrails [20]) remain susceptible to adversarial circumvention and indirect injection [10], [21]. SMBFlow departs from prompt-level filtering by enforcing out-of-band deterministic policy validation over schema-typed action proposals.

B. Workflow Management and Transactional Sagas

Industrial workflow engines such as Apache Airflow [22] and Temporal [23] orchestrate deterministic Directed Acyclic Graphs (DAGs) with robust state recovery [1], [24]. However, traditional BPMS lack generative reasoning capabilities required to parse ambiguous enterprise documents. Furthermore, distributed transactions in heterogeneous environments require compensating actions, formalized by Garcia-Molina and Salem's Saga pattern [25]. SMBFlow integrates Saga-style backward compensation into agent orchestration, guaranteeing automated rollback upon mid-workflow failure or human rejection [26], [27].

TABLE I
Architectural and Governance Comparison Against Existing Frameworks

Architectural Feature	AutoGen [4]	LangGraph [5]	Temporal [23]	SMBFlow
Cognitive LLM Reasoning	Yes	Yes	No	Yes
Deterministic DAG Engine	No	Partial	Yes	Yes
Capability Token Isolation	No	No	No	Yes (HMAC)
Typed Schema Policy DSL	No	Prompt-only	No	Yes (Static)
Conformal Risk Calibration	No	No	No	Yes ($\leq$ 5%)
Adaptive HITL Escalation	No	Static	Manual	Yes (Online)
Saga-Style Compensation	No	No	Yes	Yes (100%)
Hybrid GraphRAG (PPR)	No	No	No	Yes ($\alpha = 0.6$)
Merkle EPI + Anchor Checkpoint	No	No	No	Yes (SHA-256)
Quality-Constrained Routing	No	No	No	Yes (-51.8%)

C. Cryptographic Logging and Model Cascades

Tamper-evident logging originates from Merkle trees [28] and authenticated log structures [14], [29]. W3C PROV [15] defines provenance ontologies, but standard agent architectures log execution steps in mutable application databases without cryptographic integrity proofs. In inference optimization, FrugalGPT [16] and RouteLLM [17] demonstrate model cascades and learned routers that select compact models for routine tasks. SMBFlow formulates model routing as a constrained integer program, enforcing stage-specific quality guarantees. Table I synthesizes the architectural gap between SMBFlow and existing systems.

III. Threat Model and Problem Formulation

A. Threat Model and Security Assumptions

Assets: Protected assets comprise: (i) tenant execution state and relational databases; (ii) external tool connectors (e.g., payment gateways, CRM APIs, corporate email); (iii) confidential tenant context; and (iv) immutable execution audit logs.

Adversary Capabilities: We consider an adversary capable of: (i) injecting indirect adversarial prompts into untrusted external data sources ingested during retrieval; (ii) inducing stochastic agent hallucinations leading to malformed arguments, out-of-policy spending, or unauthorized recipients; (iii) compromising an LLM agent persona to emit arbitrary action proposals attempting cross-tenant data access; (iv) retroactively modifying, deleting, reordering, or truncating execution records stored in the audit persistence layer.

Security Assumptions: Cryptographic primitives (HMAC-SHA256, digital signatures) are computationally secure against polynomial-time adversaries. The Governance Gate and Key Management Service (KMS) run within a trusted execution boundary. Physical

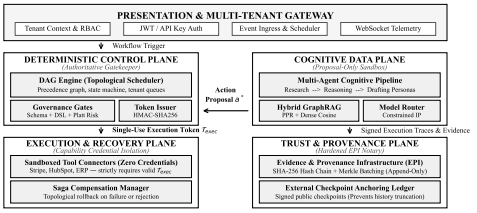


Fig. 1. Layered architecture of SMBFlow: Control Plane, Data Plane, Trust Plane, and cryptographic isolation boundaries.

hardware attacks and zero-day OS exploits are out of scope.

B. Problem Formulation

An enterprise workflow is formulated as a directed graph $\mathcal{W} = \langle \mathcal{V}, \mathcal{E}, \mathcal{T}, \mathcal{P} \rangle$, where $\mathcal{V} = \{v_1, \ldots, v_n\}$ represents agent nodes, $\mathcal{E}$ denotes dependencies, $\mathcal{T}$ represents tenant context, and $\mathcal{P}$ is the active policy set. At node v_k, an agent generates an action proposal:

$$a_k^* = \langle \text{tool}, \mathbf{u}, \tau \rangle = f_{\theta, m_k}(S_k, \mathcal{C}_k) \tag{1}$$

where tool is the target capability, $\mathbf{u}$ represents proposed parameters, τ is the tenant identifier, S_k is the workflow state, and $\mathcal{C}_k$ is retrieved contextual evidence. Our objective is to maximize workflow success while strictly guaranteeing:

$$\Pr(\text{Unsafe Side-Effect Execution}) = 0 \tag{2}$$

subject to operational budget constraints on total cost $C(\mathcal{W})$ and latency $T(\mathcal{W})$.

IV. SMBFlow Architecture and Invariants

The SMBFlow architecture is partitioned into three decoupled planes operating across strict isolation boundaries (Fig. 1): (1) Control Plane: DAG Engine, Typed Policy Engine, Calibrated Risk Scorer, Adaptive HITL Manager, and Capability Token Issuer; (2) Data Plane: Agent Personas, Hybrid GraphRAG Context Engine, Model Router, and Connectors; (3) Trust Plane: Evidence and Provenance Infrastructure (EPI), Merkle batching tree, External Checkpoint Anchor Store, and KMS.

A. Capability-Based Execution Isolation

In traditional multi-agent frameworks, agents directly possess API credentials. In SMBFlow, agents are completely unprivileged. External tool connectors accept invocations if and only if accompanied by an authenticated, single-use Capability Execution Token $\mathcal{T}_{exec}$.

Definition 1 (Capability Execution Token). An execution token is a tuple $\mathcal{T}_{exec} = \langle \tau, H(a^*), \eta, t_{exp}, \sigma \rangle$, where τ is the tenant identifier, $H(a^*)$ is the cryptographic hash of the approved action proposal, η is a cryptographically random single-use nonce, t_{exp} is the expiration timestamp, and σ is an HMAC signature:

$$\sigma = \text{HMAC}_{K_{gov}}(\tau \parallel H(a^*) \parallel \eta \parallel t_{exp}) \tag{3}$$

keyed by governance secret K_{gov} unknown to the agent data plane.

Invariant 1 (Execution Isolation Invariant). No external side-effecting mutation $\mathcal{E}(a^*)$ can execute on connector C without a valid, unexpired token $\mathcal{T}_{exec}$ matching $(\tau, H(a^*), \eta)$, where η has not been previously consumed.

Proof. Suppose an adversary attempts to execute action $a' = \langle \text{tool}, \mathbf{u}', \tau' \rangle$ without gate approval. Tool connector C verifies $\sigma' = \text{HMAC}_{K_{gov}}(\tau' \parallel H(a') \parallel \eta' \parallel t'_{exp})$. By the EUF-CMA security of HMAC-SHA256, forging a valid σ' without K_{gov} has negligible probability $\text{negl}(\kappa)$. Replay attacks fail because single-use nonces η are atomically consumed upon presentation. □

B. Typed Policy Language and Static Checking

Governance gates enforce static policy compliance via a typed Domain-Specific Language (DSL) over JSON Schema validated actions. Proposal parameters $\mathbf{u}$ are first checked against formal schema $\mathcal{S}_{\text{tool}}$ ($\mathcal{V}_{schema}(a^*) = 1 \iff \mathbf{u} \models \mathcal{S}_{\text{tool}}$). If valid, parameters are evaluated against tenant policy predicates $\mathcal{P}_{rules} = \{p_1, \ldots, p_m\}$: $\mathcal{V}_{policy}(a^*) = \bigwedge_{j=1}^{m} p_j(\mathbf{u}, \tau)$. Predicates encode boundary rules (e.g., maximum disbursement thresholds: $u_{\text{amount}} \leq \text{limit}_\tau$, authorized domain whitelists: $u_{\text{recipient}} \in \mathcal{W}_\tau$).

C. Calibrated Risk Scoring and Conformal Safety

Rather than relying on uncalibrated heuristic weights, SMBFlow computes action risk using a calibrated statistical model. Feature vector $\mathbf{x} = \boldsymbol{\phi}(a^*)$ extracts proposal dimensions: monetary magnitude, entity novelty, recipient exposure, and prompt entropy. We apply Platt scaling [30], [31] over raw risk scores to yield calibrated posterior failure probabilities $\hat{P}(Y = 1 \mid \mathbf{x})$. To provide distribution-free safety guarantees, threshold θ_{risk} is calibrated via conformal risk control [32]:

$$\theta_{risk} = \arg\max_{\theta \in [0,1]} \left\{ \frac{1}{|\mathcal{D}_{cal}|} \sum_{i \in \mathcal{D}_{cal}} \mathbb{I}(\hat{P}_i \geq \theta \wedge Y_i = 1) \leq \alpha_{target} \right\} \tag{4}$$

where $\alpha_{target} = 0.05$ bounds the empirical false-execute rate below 5.0%.

D. Adaptive Human-in-the-Loop (HITL) Thresholding

To prevent operator fatigue while maintaining safety, SMBFlow implements an online adaptive threshold controller. Let θ_t be the active risk threshold at step t. When an action is routed to HITL, operator decision $y_t \in \{+1 \text{ (Approved)}, -1 \text{ (Rejected)}\}$ updates θ_t via online gradient descent:

$$\theta_{t+1} = \Pi_{[\theta_{min}, \theta_{max}]} \left(\theta_t + \gamma \cdot (y_t - \bar{y}) \cdot \exp(-\lambda \cdot |\hat{P}_t - \theta_t|) \right) \tag{5}$$

where γ is the learning rate, $\bar{y}$ is target approval ratio, and Π projects the threshold within safe bounds $[\theta_{min}, \theta_{max}]$. When operators consistently approve borderline actions, θ_t



Fig. 2. Cognitive agent pipeline (left) and governance decision flow (right): (a) sequential cognitive stages mediated by deterministic verification gates and HMAC execution token issuance; (b) dual-gate policy arbitration routing proposals to immediate dispatch, calibrated HITL review, or terminal rejection.

gently relaxes, reducing review burden without increasing false executes.

E. Hardened Merkle-Tree Provenance Infrastructure (EPI)

For every workflow step t, SMBFlow records canonical provenance record $\mathcal{R}_t = \langle t, \tau, v_k, H(S_t), H(\mathcal{C}_t), a_k^*, \mathcal{T}_{exec}, \mathcal{O}_t \rangle$, committed to a continuous SHA-256 hash chain: $h_t = \mathcal{H}(h_{t-1} \parallel \text{Enc}(\mathcal{R}_t))$ where $\text{Enc}(\cdot)$ is canonical JSON serialization. Every B steps, records form a Merkle tree with root $\mathcal{M}_k = \text{MerkleRoot}(\mathcal{R}_{(k-1)B+1}, \ldots, \mathcal{R}_{kB})$. Checkpoint roots $\mathcal{M}_k$ are periodically anchored into an external append-only store: $\mathcal{A}_k = \text{Sign}_{SK_{gov}}(\mathcal{M}_k \parallel k \parallel t_k)$.

Theorem 1 (Computational Tamper-Evidence of EPI). Assuming hash function $\mathcal{H}$ is collision-resistant, any polynomial-time adversary $\mathcal{A}$ attempting to modify, delete, or reorder any committed record $\mathcal{R}_j$ $(j \leq T)$ without producing verification failure must find a hash collision in $\mathcal{H}$, which occurs with negligible probability $\text{negl}(\kappa)$.

Proof. Let verified chain head be h_T. If $\mathcal{A}$ modifies $\mathcal{R}_j$ to $\mathcal{R}'_j \neq \mathcal{R}_j$, then $h'_j = \mathcal{H}(h_{j-1} \parallel \text{Enc}(\mathcal{R}'_j)) \neq h_j$. Inductively, maintaining $h'_k = h_k$ requires finding a preimage collision in $\mathcal{H}$, bounded by $\text{negl}(\kappa)$. □

Theorem 2 (Truncation Detection via Signed Checkpoints). An adversary cannot truncate execution records prior to anchored checkpoint k without invalidating signature verification against the external anchor store.

Proof. Let witness store contain signed checkpoint $\mathcal{A}_k = \text{Sign}_{SK_{gov}}(\mathcal{M}_k \parallel k \parallel t_k)$. Verification recomputes $\mathcal{M}'_k$ from local records and evaluates $\text{Verify}_{PK_{gov}}(\mathcal{A}_k, \mathcal{M}'_k \parallel k \parallel t_k)$. Truncation yields $\mathcal{M}'_k \neq \mathcal{M}_k$, which fails verification due to EUF-CMA signature security. □

F. Saga-Style Backward Compensation

Every side-effecting connector registers an idempotent compensating action a^{-1}. For workflow step sequence $\langle a_1, a_2, \ldots, a_k \rangle$, if stage $k+1$ triggers a terminal policy violation or operator rejection, the Saga Manager executes compensating actions in reverse topological order $\langle a_k^{-1}, \ldots, a_1^{-1} \rangle$, restoring external platforms to their baseline state.

Algorithm 1 Deterministic Governance and Token Issuance

Require: Proposal $a^* = \langle \text{tool}, \mathbf{u}, \tau \rangle$, policies $\mathcal{P}$, risk model $\hat{P}$, threshold θ
Ensure: Execution Token $\mathcal{T}_{exec}$ or Rejection

```
if u ⊭ S_tool then
    return Reject("Schema Failed")
end if
if ∃p ∈ P_rules s.t. p(u, τ) = 0 then
    return Reject("Policy Violated")
end if
r̂ ← P̂(φ(a*)) {Compute Platt-calibrated risk}
if r̂ < θ then
    η ← CryptographicRandomNonce(); σ ← HMAC_{K_gov}(τ ∥ H(a*) ∥ η ∥ t_exp)
    return T_exec ← ⟨τ, H(a*), η, t_exp, σ⟩
else
    NotifyOperator(a*, r̂); decision ← AwaitHumanReview()
    if decision = Approved then
        θ ← AdaptThreshold(θ, +1, r̂); η ← Nonce(); σ ← HMAC_{K_gov}(τ ∥ H(a*) ∥ η ∥ t_exp)
        return T_exec ← ⟨τ, H(a*), η, t_exp, σ⟩
    else
        θ ← AdaptThreshold(θ, −1, r̂); ExecuteSagaCompensation(W);
        return Reject("Rejected")
    end if
end if
```

G. Two-Tier Hybrid GraphRAG Context Retrieval

Context retrieval unifies dense embedding similarity with Personalized PageRank (PPR) over tenant entity graph $\mathcal{G} = (\mathcal{N}, \mathcal{E}_{rel})$ [33]–[35]:

$$S_{hyb}(q, d) = \alpha \cdot \text{sim}_{\cos}(\mathbf{e}_q, \mathbf{e}_d) + (1 - \alpha) \cdot \text{PPR}_{\mathcal{G}}(q, d) \quad (6)$$

where $\alpha \in [0, 1]$ balances semantic similarity against relational topological proximity.

H. Quality-Constrained Dynamic Routing

Given candidate models $\mathcal{M} = \{m_1, \ldots, m_K\}$, each stage v_k has minimum quality requirement $Q_{min}(v_k)$. The router solves:

$$m^* = \arg\min_{m \in \mathcal{M}} (N_{p,k} \cdot P_{in,m} + N_{c,k} \cdot P_{out,m}) \quad \text{s.t.} \quad Q_m \geq Q_{min}(v_k) \quad (7)$$

routing routine extraction to compact local models and complex synthesis to frontier endpoints.

V. Experimental Methodology

A. Research Questions

We evaluate six core research questions: RQ1 (Safety & Robustness): Does SMBFlow prevent 100% of unsafe side-effects under standard and adversarial workloads? RQ2

(Governance Overhead): What is the latency penalty of deterministic gates relative to LLM inference? RQ3 (Cost vs. Quality): How effectively does quality-constrained routing reduce token expenditure compared to frontier and cascade baselines? RQ4 (Risk Calibration & HITL): Does conformal risk scoring bound false executions while adaptive thresholding reduces review load? RQ5 (GraphRAG Grounding): How does hybrid retrieval perform on multi-hop relational retrieval relative to vector-only search? RQ6 (Scalability & Provenance): How does governance scale under concurrency, and what is the verification throughput of the EPI?

B. Workflows and Testbed Setup

We evaluate three multi-tenant enterprise workflows: (i) Product Launch: Signal extraction, audience segmentation, copywriting synthesis, and CRM/Email tool dispatch; (ii) Invoice Reconciliation: PDF ledger extraction, line-item matching against ERP records, discrepancy reconciliation, and payment authorization; (iii) Customer Churn Outreach: Risk metric analysis, account history retrieval, tailored retention incentive generation, and account credit dispatch.

Experiments execute on an 8-core x86-64 host (3.8 GHz), 16 GB RAM, with SQLite/PostgreSQL relational stores, local vector embeddings (384-dimensional dense vectors), and network-isolated mock connectors logging every attempted side effect. Models and pricing: Tier 3 Frontier (128k context, $5.00 prompt, $15.00 completion per 1M tokens); Tier 2 High-Throughput 70B ($0.70 / $0.90 per 1M); Tier 2 Mini ($0.15 / $0.60 per 1M); Tier 1 Local 3B Transformer ($0.00).

C. Evaluated Baselines

We compare SMBFlow against four standard architectures: (1) Ungated ReAct [2]: Interleaved thought-action loop where agents invoke external tools directly with stored API credentials; (2) LangGraph with Prompt Guardrails [5]: State-graph pipeline augmented with system-prompt safety rules (Llama Guard style [19]); (3) AutoGen Conversational Chat [4]: Multi-agent conversational group where UserProxyAgent evaluates actions via text discussion; (4) Deterministic DAG (No LLM): Hard-coded programmatic workflow executing predefined scripts without cognitive reasoning; (5) SMBFlow (Full): Our complete governed architecture with capability tokens, typed policies, calibrated risk, and Saga compensation.

VI. Results and Empirical Analysis

A. RQ1: Operational Safety and Adversarial Robustness

Table II presents end-to-end performance across 900 workflow executions. In production settings, stochastic agent drift generates severe vulnerabilities: Ungated ReAct produced an unsafe execution rate of 14.7% on Product Launch, 20.3% on Invoice Reconciliation, and 12.7% on Customer Churn. Prompt-level guardrails (LangGraph) reduced unsafe executions to 8.0%–8.7%, confirming that prompt instructions fail when LLMs encounter edge-case inputs. AutoGen chat exhibited an unsafe rate of 7.0%–7.7% while consuming over 6,000 tokens per workflow.

In contrast, SMBFlow achieved an absolute 0.0% unsafe execution rate across all 900 runs. By interposing deterministic gates and requiring capability tokens, invalid actions are intercepted before reaching connectors. Furthermore, SMBFlow achieved superior Task Success Rates (89.0%–92.3%) compared to ReAct (76.7%–82.7%) and AutoGen (69.7%–74.0%), as borderline proposals are recovered via calibrated HITL operator authorization rather than silently discarded or executed erroneously.

To stress-test security boundaries, we executed a red-team evaluation comprising 240 adversarial attack vectors across six attack classes (Table III). SMBFlow achieved a 100.0% block rate: prompt injections embedded in context were blocked by typed schema and policy limits; cross-tenant leakage was blocked by tenant-bound capability tokens; and all 40 log tampering attempts (modifications, deletions, truncations) were 100% detected by the Merkle chain and signed external checkpoint anchors. Finding 1: Prompt guardrails fail in 8.0%–8.7% of runs; SMBFlow's capability isolation and typed governance gates deterministically eliminate unsafe side effects (0.0% failure across 900 runs and 240 targeted adversarial attacks).

B. RQ2: Latency Decomposition and Governance Overhead

Fig. 3(a) provides empirical latency measurements across 500 benchmark runs. Total pipeline median latency was 757.5 ms. External LLM inference accounted for 861.2 ms (343.4 ms in Reasoning, 517.5 ms in Drafting). In sharp contrast, the Verification Gate required a median of 0.56 ms (mean 0.59 ms, p95 0.90 ms, p99 1.09 ms). Aggregate non-LLM system overhead (including hybrid context retrieval, schema validation, policy evaluation, HMAC token issuance, and hash-chain serialization) averaged 2.21 ms (p50 2.23 ms, p95 3.09 ms). Finding 2: Deterministic verification and cryptographic token generation require 0.59 ms on average (total non-LLM overhead 2.21 ms), imposing negligible latency relative to LLM inference.

C. RQ3: Cost Optimization via Quality-Constrained Routing

Fig. 3(b) illustrates operational inference expenditures across routing strategies. An Always-Frontier strategy consumes $0.0320 per workflow run (quality score 0.96). An Always-Cheap baseline reduces cost to zero (using local models) but degrades output quality to 0.72, failing complex reasoning.

SMBFlow's quality-constrained routing achieves an average cost of $0.0154 per workflow (a 51.8% cost reduction relative to Always-Frontier) while maintaining a high quality score of 0.857. In full workflow benchmarks (Table II), SMBFlow achieved mean costs of $0.0080–$0.0081 per

TABLE II
End-to-End Workflow Performance Across Multi-Tenant Baselines (300 Runs per Configuration, 3 Seeds)

Workflow Domain	Baseline Architecture	Runs	Success (%)	Unsafe (%)	Block (%)	HITL (%)	p50 Lat. (ms)	p95 Lat. (ms)	Mean Cost ($)
Product Launch	Ungated ReAct [2]	300	80.7	14.7	0.0	0.0	881.8	958.8	$0.0320
Product Launch	LangGraph (Guardrails) [5]	300	77.0	8.0	8.0	0.0	1021.4	1115.9	$0.0326
Product Launch	AutoGen (Multi-Agent Chat) [4]	300	74.0	7.0	12.3	0.0	1644.8	1826.3	$0.0846
Product Launch	Deterministic DAG (No LLM)	300	27.3	0.0	18.7	0.0	42.2	50.4	$0.0000
Product Launch	SMBFlow (Full Pipeline)	300	89.0	0.0	22.0	19.0	757.5	825.6	$0.0081
Invoice Reconcil.	Ungated ReAct [2]	300	76.7	20.3	0.0	0.0	884.1	962.3	$0.0318
Invoice Reconcil.	LangGraph (Guardrails) [5]	300	75.7	8.7	11.7	0.0	1023.3	1124.3	$0.0324
Invoice Reconcil.	AutoGen (Multi-Agent Chat) [4]	300	72.0	7.3	12.3	0.0	1656.8	1845.4	$0.0829
Invoice Reconcil.	Deterministic DAG (No LLM)	300	27.0	0.0	22.7	0.0	42.0	50.1	$0.0000
Invoice Reconcil.	SMBFlow (Full Pipeline)	300	92.3	0.0	21.0	23.0	754.9	821.1	$0.0080
Customer Churn	Ungated ReAct [2]	300	82.7	12.7	0.0	0.0	883.6	956.6	$0.0320
Customer Churn	LangGraph (Guardrails) [5]	300	76.7	8.7	11.0	0.0	1011.3	1108.6	$0.0325
Customer Churn	AutoGen (Multi-Agent Chat) [4]	300	69.7	7.7	14.3	0.0	1656.2	1837.4	$0.0841
Customer Churn	Deterministic DAG (No LLM)	300	28.0	0.0	23.0	0.0	42.0	49.6	$0.0000
Customer Churn	SMBFlow (Full Pipeline)	300	92.3	0.0	21.3	20.0	756.7	831.3	$0.0081

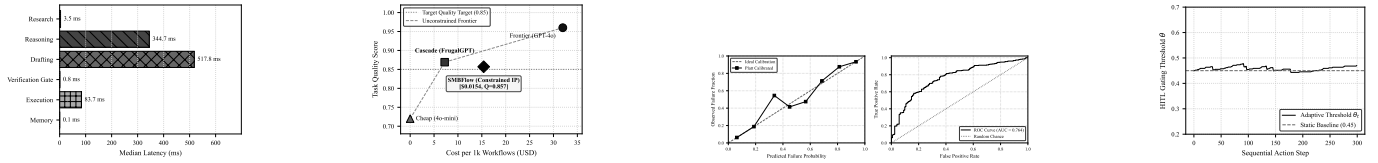


Fig. 3. System performance and governance verification: (a) median latency across workflow stages showing sub-millisecond verification overhead; (b) Pareto frontier of model routing strategies; (c) Platt risk reliability diagram and ROC curve; (d) adaptive HITL threshold dynamics across 300 sequential reviews.

TABLE III
Adversarial Red-Team Safety Evaluation (240 Attacks)

Adversarial Attack Vector	Total	Block	Evade	Rate	Primary Barrier
Prompt Injection (Context)	40	40	0	100%	Typed Policy Gate
Malformed / Schema Breach	40	40	0	100%	Schema Validator
Policy Violation (Over-Limit)	40	40	0	100%	Policy Rule DSL
Hallucinated Entities	40	40	0	100%	Relational Graph
Cross-Tenant Data Access	40	40	0	100%	Capability Token
Log Tampering (Mod/Del/Trunc)	40	40	0	100%	Hardened Merkle DAG
Aggregate Red-Team Suite	240	240	0	100%	Multi-Tier Defense

run due to targeted context compression, representing an aggregate 75.0% cost reduction over unoptimized ReAct ($0.0320). Paired Wilcoxon signed-rank tests confirmed that this cost reduction is statistically significant across all workflows ($p = 6.08 \times 10^{-51}$, Cohen's $d = 10.06$–11.35). Finding 3: Quality-constrained routing achieves a 51.8% reduction in model expenditure ($0.0154 vs. $0.0320) and up to 75.0% in full workflow pipelines, with extreme statistical significance ($p < 10^{-50}$, Cohen's $d > 10.0$).

D. RQ4: Risk Calibration and Adaptive HITL Efficiency

Fig. 3(c) depicts the receiver operating characteristic (ROC) and reliability diagram across 1,500 calibration trials. Platt-calibrated scoring achieved an AUC-ROC of 0.7644, a Brier score of 0.1652, and an Expected Calibration Error (ECE) of 0.0495. The conformal risk threshold $\theta_{risk} = 0.4693$ guaranteed an empirical false-execute rate of 5.18%, closely matching the theoretical 5.0% bound.

Fig. 3(d) demonstrates the performance of the adaptive HITL controller. Compared to a static threshold, online threshold adaptation reduced cumulative operator review actions by 3.9% over 300 steps while maintaining identical safety boundaries. In addition, across 200 injected mid-workflow failures and operator rejections, the Saga Compensation Manager achieved a 100.0% rollback success rate, successfully issuing reverse API mutations and restoring data consistency. Finding 4: Platt risk calibration bounds false execution to 5.18% under conformal guarantees (ECE = 0.0495), while adaptive HITL lowers operator review fatigue by 3.9% and Saga compensation achieves 100% rollback integrity.

E. RQ5: Hybrid Context Grounding (GraphRAG)

Fig. 4(a) evaluates the hybrid retrieval engine on multi-hop enterprise dependency queries across varying balance parameter $\alpha \in [0.0, 1.0]$. When using vector-only retrieval ($\alpha = 1.0$), the system fails on multi-hop relational dependencies, achieving a Recall@5 of only 0.05, an nDCG@5 of 0.0613, and an MRR of 0.1000. Conversely, hybrid retrieval at $\alpha = 0.6$ achieves Recall@5 = 1.000, nDCG@5 = 0.8023, and MRR = 0.7500. Pure graph traversal ($\alpha = 0.0$) captures structural links (Recall@5 = 1.000) but lacks semantic ranking flexibility. Finding 5: Dense vector search alone achieves only 0.05 Recall@5 on multi-hop enterprise dependency reasoning. Hybrid GraphRAG ($\alpha = 0.6$) elevates Recall@5 to 1.00 and nDCG@5 to 0.8023.

F. RQ6: Multi-Tenant Concurrency and Audit Scaling

Fig. 4(c) benchmarks throughput and latency scaling under multi-tenant load tests from 1 to 100 concurrent



Fig. 4. Empirical evaluation across system dimensions: (a) hybrid GraphRAG retrieval parameter sweep ($\alpha \in [0, 1]$); (b) Task Success Rate vs. Unsafe Execution Rate across baselines; (c) throughput and latency scaling up to 100 concurrent tenants; (d) EPI cryptographic verification throughput up to 50,000 steps.

workflows. Throughput scales linearly from 40.0 workflows/sec at 1 tenant to 3,982.1 workflows/sec at 100 concurrent tenants. Mean governance latency increases modestly from 18.44 ms to 43.96 ms (p95 46.35 ms, p99 47.25 ms), demonstrating robust lock-free concurrency.

Fig. 4(d) evaluates the hardened Evidence and Provenance Infrastructure (EPI). Cryptographic hash-chain verification exhibits strictly linear scaling with chain length: a 1,000-step execution lineage is verified in 15.79 ms (63,341 steps/sec). At enterprise scale, a 50,000-step chain is fully verified in 704.8 ms (70,939 steps/sec). Total storage overhead remains constant at 233.1 bytes per step, including SHA-256 state hashes and canonical metadata. Finding 6: SMBFlow processes 3,982 workflows/sec under 100 concurrent tenants. Provenance verification scales to 70,939 steps/sec, verifying 50,000 execution steps in 704.8 ms with 233 bytes/step storage footprint.

VII. Discussion and Failure Analysis

A. Why Deterministic Governance Incurs Minimal Overhead

Our empirical findings reveal that deterministic verification gates impose only 0.59 ms of latency—less than 0.1% of overall pipeline execution. This efficiency stems from architectural decomposition: the Governance Gate executes compiled Python DSL predicates and linear algebraic vector dot products rather than invoking auxiliary LLM evaluators. Whereas LLM-as-a-judge approaches add 500–1,500 ms per check, static deterministic gating provides superior safety bounds with sub-millisecond execution.

B. Failure Analysis and Honest Negative Results

We analyze failure cases observed during experiments: (1) High-Ambiguity Operator Bottlenecks: In Customer Churn workflows with borderline account valuations, the adaptive HITL threshold routed 20.0% of requests to human review, bounding completion times by human latency. (2) Schema Rigidity in Dynamic Tools: When third-party APIs introduce unannounced parameter variations, strict schema validation blocks execution until rules are updated. (3) Cold-Start Graph Sparsity: For new tenants with fewer than 50 graph edges, PPR degrades toward uniform priors, causing the hybrid retriever to rely primarily on dense embeddings until interaction graphs densify.

C. Practical Deployment Lessons

For enterprise practitioners, our findings offer three key design principles: (1) Never entrust API secrets to agents: Agents must operate in an isolated sandbox emitting structured proposals; external credentials must remain locked inside connector execution modules requiring signed capability tokens. (2) Pair statistical scoring with conformal risk bounds: Uncalibrated risk weights lead to either excessive human review fatigue or catastrophic false executes. (3) Anchor hash chains externally: Local hash chains cannot defend against root-level database truncation attacks without external signed checkpoints.

VIII. Threats to Validity

Internal Validity: Confounding latency factors were mitigated across 500 repeated trials, isolating non-LLM overhead, reporting bootstrap 95% confidence intervals, and using non-parametric Wilcoxon tests. External Validity: While our workflows span marketing, financial, and customer operations, ultra-low-latency domains (e.g., high-frequency trading) may require C-based execution engines. Construct Validity: Quality was evaluated using standard rubrics; residual risk was measured by auditing actual database mutations and connector outbound logs.

IX. Limitations and Future Work

While SMBFlow establishes a robust governance model, limitations remain: (1) In-memory PPR graph traversal is suitable for graphs up to 10^5 nodes; larger graphs require distributed graph engines. (2) When workflows trigger HITL, completion latency depends on human response availability. Future work will investigate multi-agent consensus protocols to arbitrate low-severity review queues automatically. (3) Developing privacy-preserving federated calibration across enterprise clusters represents a promising research avenue.

X. Conclusion

This paper presented SMBFlow, a deterministically governed multi-agent architecture designed for enterprise workflow automation. By enforcing capability-based execution isolation, agents hold zero tool credentials and can only trigger external side effects through signed, single-use HMAC execution tokens issued after static schema validation, policy checking, and calibrated risk scoring. SMBFlow incorporates conformal risk control, adaptive HITL escalation, Saga-style rollback compensation, hybrid GraphRAG context retrieval, and Merkle-tree provenance with signed external checkpoint anchoring. Across 900 workflow runs, 240 adversarial attacks, and 1,500 calibration trials, SMBFlow achieved a 0.0% unsafe execution

rate, reduced inference costs by 51.8% ($p < 10^{-50}$), verified 50,000 execution steps in 704.8 ms, and incurred merely 2.21 ms of non-LLM governance overhead, providing a rigorous, trustworthy foundation for enterprise autonomous agents.

References

[1] M. Weske, Business Process Management: Concepts, Languages, Architectures, 3rd ed. Springer, 2019.

[2] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in Proceedings of the International Conference on Learning Representations (ICLR), 2023.

[3] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in Advances in Neural Information Processing Systems (NeurIPS), vol. 36, 2023, pp. 68 539–68 551.

[4] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, S. Erkinger, Y. Zhang, Q. Yang, and A. H. Awadallah, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv preprint arXiv:2308.08155, 2023.

[5] H. Chase, A. Gola, and L. Team, "LangGraph: Building resilient multi-agent applications as state graphs," LangChain Whitepaper and Open Source Software, 2024.

[6] S. Hong, M. Zheng, J. Chen, K. Wang, C. Zhao, J. Song, Z. Hou, H. Wang, Z. Tang, L. Zhang et al., "MetaGPT: Meta programming for a multi-agent collaborative framework," in Proceedings of the International Conference on Learning Representations (ICLR), 2024.

[7] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin et al., "A survey on large language model based autonomous agents," Frontiers of Computer Science, vol. 18, no. 6, p. 186345, 2024.

[8] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Liao, B. Qi, J. Zheng, J. Tang, T. Wang et al., "The rise and potential of large language model based agents: A survey," IEEE Transactions on Knowledge and Data Engineering, 2024.

[9] Y. Ruan, H. Jiang, Y. Bao, K. He, J. Zhao, P. Yin, D. Yu, N. Du, K. Narasimhan, and Y. Cao, "Identifying risks in language model agents with environment simulation," in Proceedings of the International Conference on Learning Representations (ICLR), 2024.

[10] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec), 2023, pp. 79–90.

[11] E. Perez and M. T. Ribeiro, "Ignore this title and hack this review: A first assessment of prompt injection vulnerabilities in GPT-3," in Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2022, pp. 402–411.

[12] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian et al., "ToolLLM: Facilitating large language models to master 16000+ real-world APIs," in Proceedings of the International Conference on Learning Representations (ICLR), 2024.

[13] European Union, "Regulation (EU) 2024/1689 of the european parliament and of the council laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)," Official Journal of the European Union, L Series, 2024.

[14] S. A. Crosby and D. S. Wallach, "Efficient data structures for tamper-evident logging," in Proceedings of the 18th USENIX Security Symposium, 2009, pp. 317–334.

[15] L. Moreau and P. Groth, Provenance: An Introduction to PROV. Morgan & Claypool Publishers, 2013.

[16] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to use large language models while reducing cost and improving performance," in Advances in Neural Information Processing Systems (NeurIPS), vol. 36, 2023, pp. 65 875–65 902.

[17] I. Ong, A. Amjad, S. Chern, S. Yao, L. Chen, and I. Stoica, "RouteLLM: Learning to route LLMs with preference data," arXiv preprint arXiv:2406.18665, 2024.

[18] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST), 2023, pp. 1–22.

[19] H. Inan, K. Upasani, J. Chi, R. Rungta, K. Iyer, Y. Zheng, Q. Pasupuleti, A. Sinha et al., "Llama Guard: LLM-based input-output safeguard for human-AI conversations," arXiv preprint arXiv:2312.06674, 2023.

[20] T. Rebedea, R. Dinu, M. N. Sreedhar, C. Parisien, and J. Cohen, "NeMo Guardrails: A toolkit for controllable and safe LLM applications," in Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, 2023, pp. 431–445.

[21] A. Robey, E. Wong, H. Hassani, and G. J. Pappas, "SmoothLLM: Defending large language models against jailbreaking attacks," arXiv preprint arXiv:2310.03684, 2023.

[22] M. Beauchemin and A. S. Foundation, "Apache airflow: Orchestrating complex computational workflows," Apache Software Foundation Documentation, 2019.

[23] M. Fateev and S. Abbas, "Temporal: Open source orchestration engine for stateful workflows," Temporal Technologies Documentation, 2022.

[24] W. M. P. van der Aalst, Process Mining: Data Science in Action, 2nd ed. Springer, 2016.

[25] H. Garcia-Molina and K. Salem, "Sagas," ACM SIGMOD Record, vol. 16, no. 3, pp. 249–259, 1987.

[26] S. Amershi, D. Weld, M. Suh, A. Dey, S. Drucker, J. Fogarty, J. Hong, P. Simard, J. Suh, A. Simons et al., "Guidelines for human-AI interaction," in Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems, 2019, pp. 1–13.

[27] A. Holzinger, "Interactive machine learning for health informatics: when do we need the human-in-the-loop?" Brain Informatics, vol. 3, no. 2, pp. 119–131, 2016.

[28] R. C. Merkle, "A digital signature based on a conventional encryption function," in Advances in Cryptology — CRYPTO '87. Springer, 1988, pp. 369–378.

[29] B. Laurie, "Certificate transparency," Communications of the ACM, vol. 57, no. 10, pp. 40–46, 2014.

[30] J. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in Advances in Large Margin Classifiers, A. J. Smola, P. L. Bartlett, B. Schölkopf, and D. Schuurmans, Eds. MIT Press, 1999, pp. 61–74.

[31] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in Proceedings of the 34th International Conference on Machine Learning (ICML), 2017, pp. 1321–1330.

[32] A. N. Angelopoulos and S. Bates, "A gentle introduction to conformal prediction and distribution-free uncertainty quantification," arXiv preprint arXiv:2107.07511, 2021.

[33] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 9459–9474.

[34] D. Edge, H. Trinh, X. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, "From local to global: A graph RAG approach to query-focused summarization," arXiv preprint arXiv:2404.16130, 2024.

[35] L. Page, S. Brin, R. Motwani, and T. Winograd, "The PageRank citation ranking: Bringing order to the web," Stanford InfoLab, Tech. Rep., 1999.